GraphANN: Exploring Improvements to Graph-Based Approximate Nearest Neighbor Search

GraphANN Report

M M Pruthvi Raj (DA24B016) Aakash (DA24B028) Sreekar (DA24B030)

Abstract. We report intermediate findings from our exploration of improvements to the Vamana graph-based ANN index, evaluated on the SIFT1M benchmark. Starting from the reference GraphANN implementation, we (i) incrementally refined the Vamana search and build algorithms (medoid start, two-pass refinement, catastrophic forgetting, multi-start, concurrent search optimization), (ii) designed a **Radial Layer Graph (RLG)** that enforces geometric shell coverage during pruning, (iii) built an **Inverted Voronoi Routing Graph (IVRG)** that seeds queries from the nearest Voronoi cell representative, and (iv) evaluated a **Johnson–Lindenstrauss projection variant (Vamana-JL)** that reduces build-time distance cost via random projection. **IVRG is the strongest result**, reducing average distance computations by 14–26% at matched recall versus the final Vamana baseline.

1. Introduction and Motivation

Approximate Nearest Neighbor (ANN) search underpins modern retrieval systems, recommender engines, and retrieval-augmented generation (RAG) pipelines [6]. The challenge is achieving high recall with a fraction of the $O(nd)$ cost of exact search.

Graph-based indices — in particular Vamana/DiskANN [1] and HNSW [2] — are the current state of the art for in-memory ANN. They build a navigable directed graph and answer queries via greedy beam search. The key parameters are max degree R, beam width L at build time, and the α-RNG diversity parameter.

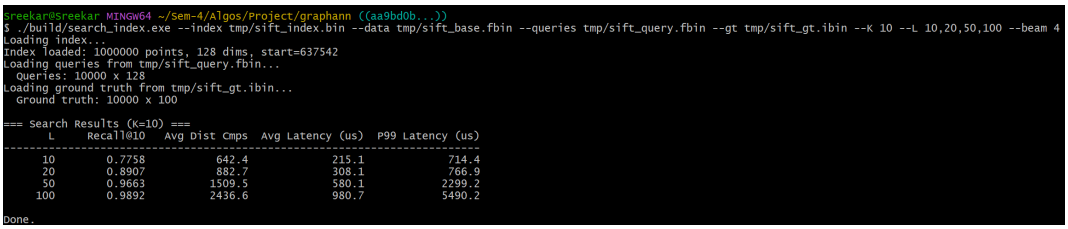
Our starting point. We are given the *GraphANN* reference implementation of Vamana (DiskANN). Our task is to propose, implement, and evaluate algorithmic improvements. We explored four directions and report results on SIFT1M (1M vectors, 128 dims, 10K queries).

2. Background: Vamana Algorithm

Vamana [1] constructs a directed graph $G = (V, E)$ where edges are selected by *robust prune*: given a candidate list C for node p, a neighbor $c \in C$ is retained only if

$$d(p, c) \leq \alpha \cdot d(c, s) \quad \forall\, s \in S_{\text{selected}}, \tag{1}$$

with $\alpha \geq 1$ controlling diversity. Search is greedy beam search from a fixed start node (the medoid), maintaining a candidate list of size L.



```
sreekar@Sreekar MINGW64 ~/Sem-4/Algos/Project/graphann ((aa9bd0b...))
$ ./build/search_index.exe --index tmp/sift_index.bin --data tmp/sift_base.fbin --queries tmp/sift_query.fbin --gt tmp/sift_gt.ibin --K 10 --L 10,20,50,100 --beam 4
Loading index...
Index loaded: 1000000 points, 128 dims, start=637542
Loading queries from tmp/sift_query.fbin...
  Queries: 10000 x 128
Loading ground truth from tmp/sift_gt.ibin...
  Ground truth: 10000 x 100

=== Search Results (K=10) ===
        L     Recall@10   Avg Dist Cmps  Avg Latency (us)  P99 Latency (us)
----------------------------------------------------------------------------
       10        0.7758           642.4             215.1             714.4
       20        0.8907           882.7             308.1             766.9
       50        0.9663          1509.5             580.1            2299.2
      100        0.9892          2436.6             980.7            5490.2

Done.
```

Figure 1: Baseline

Baseline results (GraphANN reference, random start node, single-pass build): At L=10, Recall@10 = 0.776, Avg. Dist. Cmps = 642.4.

3. Vamana Search and Build Improvements

We applied five incremental modifications to the baseline GraphANN implementation. Each was formalized mathematically and measured in isolation.

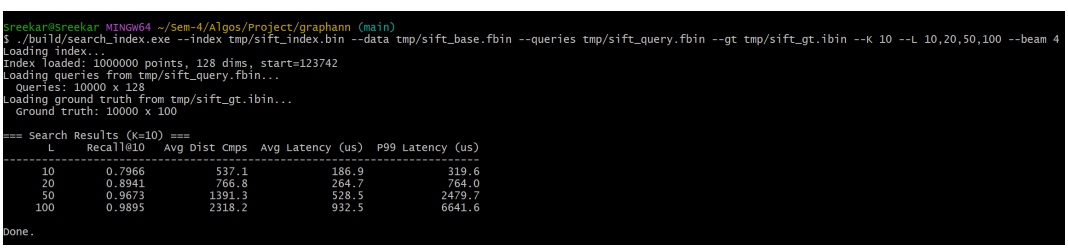
3.1. Modification 1: Medoid Start Node

The reference implementation initializes search from a random vertex $v \in V$. We replace this with the *medoid* s^*, defined as the point minimizing the L_2 distance to the dataset centroid $\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$:

$$s^* = \operatorname*{argmin}_{p \in V} \left\| x_p - \frac{1}{n}\sum_{i=1}^{n} x_i \right\|_2 . \tag{2}$$

By minimizing $\mathbb{E}[d(s^*, q)]$, we reduce the cardinality of the initial hop set $\mathcal{H}_{approach}$ required to enter the query neighborhood.

Result: Avg. Dist. Cmps decreased 642.4 → 537.1 (−16%) at L=10, with Recall@10 improved from 0.776 → 0.797.



```
sreekar@Sreekar MINGW64 ~/Sem-4/Algos/Project/graphann (main)
$ ./build/search_index.exe --index tmp/sift_index.bin --data tmp/sift_base.fbin --queries tmp/sift_query.fbin --gt tmp/sift_gt.ibin --K 10 --L 10,20,50,100 --beam 4
Loading index...
Index loaded: 1000000 points, 128 dims, start=123742
Loading queries from tmp/sift_query.fbin...
  Queries: 10000 x 128
Loading ground truth from tmp/sift_gt.ibin...
  Ground truth: 10000 x 100

=== Search Results (K=10) ===
        L     Recall@10   Avg Dist Cmps  Avg Latency (us)  P99 Latency (us)
----------------------------------------------------------------------------
       10        0.7966           537.1             186.9             319.6
       20        0.8941           766.8             264.7             764.0
       50        0.9673          1391.3             528.5            2479.7
      100        0.9895          2318.2             932.5            6641.6

Done.
```

Figure 2: Medoid Start

3.2. Modification 2: Two-Pass Refinement

To optimize the navigability of the directed graph $G(V, E)$, we implement a multi-pass construction strategy using varying diversity constraints:

- **Pass 1** ($\alpha_1 = 1.0$): Construct a base graph $\mathcal{G}_1$ enforcing strict monotonicity.
- **Pass 2** ($\alpha_2 = 1.2$): Refine $\mathcal{G}_1$ into $\mathcal{G}_2$ by allowing relaxed diversity edges.

This re-evaluates the neighbor set $\mathcal{N}(p)$ for points inserted early in the permutation π, who initially pruned against a significantly sparse set $V_{active} \subset V$.

```
Sreekar@Sreekar MINGW64 ~/Sem-4/Algos/Project/graphann (main)
$ ./build/search_index.exe --index tmp/sift_index.bin --data tmp/sift_base.fbin --queries tmp/sift_query.fbin --gt tmp/sift_gt.ibin --K 10 --L 10,20,50,100 --beam 4
Loading index...
Index loaded: 1000000 points, 128 dims, start=123742
Loading queries from tmp/sift_query.fbin...
  Queries: 10000 x 128
Loading ground truth from tmp/sift_gt.ibin...
  Ground truth: 10000 x 100

=== Search Results (K=10) ===
       L     Recall@10   Avg Dist Cmps  Avg Latency (us)  P99 Latency (us)
----------------------------------------------------------------------
      10        0.7765          477.7             171.6             304.8
      20        0.8808          676.5             231.2             371.5
      50        0.9592         1213.6             470.0            2142.3
     100        0.9850         2020.1             820.0            3984.0

Done.
```

Figure 3: Two-Pass Refinement

3.3. Modification 3: Neighborhood Merging (Catastrophic Forgetting)

During Pass 2, a standard update may replace the existing set $\mathcal{N}_1(p)$ with a disjoint set $\mathcal{N}_2(p)$, losing the high-quality connectivity established in Pass 1. We implement *neighborhood merging*:

$$\mathcal{N}_{final}(p) = \text{RobustPrune}(p, \mathcal{C}_{greedy} \cup \mathcal{N}_1(p), \alpha_2, R), \tag{3}$$

where $\mathcal{C}_{greedy}$ is the candidate set from the current search. This union operation preserves local cluster connectivity while integrating long-range diversity.

```
=== Search Results (K=10) ===
       L     Recall@10   Avg Dist Cmps  Avg Latency (us)  P99 Latency (us)
----------------------------------------------------------------------
      10        0.7954           502.9             172.2             259.4
      20        0.8947           715.3             244.2             389.1
      30        0.9348           915.0             328.8             547.2
      50        0.9679          1294.5             486.2             858.5
      75        0.9832          1740.2             703.6            3336.0
     100        0.9895          2160.5             888.8            5075.9
     150        0.9946          2945.5            1202.7            5315.2
     200        0.9967          3675.8            1612.2           10504.9

Done.
```

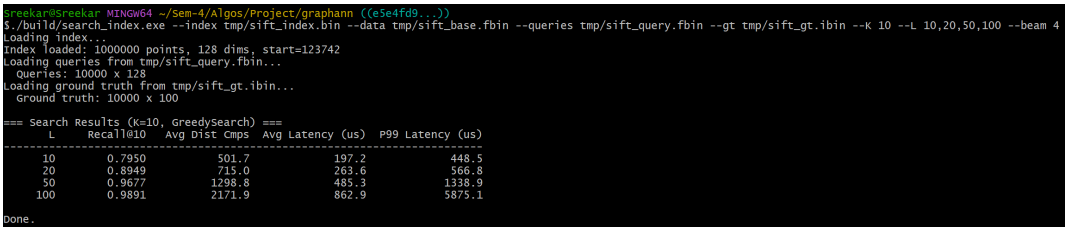
Figure 4: Catastrophic Forgetting

3.4. Modification 4: Multi-Start Beam Search (Tried, Removed)

We experimented with an ensemble of starting seeds $\mathcal{S} = \{s^*, r_1, r_2, r_3\}$, where $r_i \sim \text{Uniform}(V)$. The initial beam $\mathcal{B}_0$ is initialized as:

$$\mathcal{B}_0 = \underset{s \in \mathcal{S}}{\text{Top-L}}\{(d(q, x_s), s)\}. \tag{4}$$

While this increased Recall@10 ($0.777 \to 0.795$), it resulted in a 5% increase in $E[\text{DistCmps}]$ and introduced significant lock contention overhead in parallel search. We deferred this optimization to the IVRG adaptive seeding (Section 5).



```
Sreekar@Sreekar MINGW64 ~/Sem-4/Algos/Project/graphann ((e5e4fd9...))
$./build/search_index.exe --index tmp/sift_index.bin --data tmp/sift_base.fbin --queries tmp/sift_query.fbin --gt tmp/sift_gt.ibin --K 10 --L 10,20,50,100 --beam 4
Loading index...
Index loaded: 1000000 points, 128 dims, start=123742
Loading queries from tmp/sift_query.fbin...
  Queries: 10000 x 128
Loading ground truth from tmp/sift_gt.ibin...
  Ground truth: 10000 x 100

=== Search Results (K=10, GreedySearch) ===
       L      Recall@10   Avg Dist Cmps  Avg Latency (us)  P99 Latency (us)
----------------------------------------------------------------------------
      10        0.7950           501.7             197.2             448.5
      20        0.8949           715.0             263.6             566.8
      50        0.9677          1298.8             485.3            1338.9
     100        0.9891          2171.9             862.9            5875.1

Done.
```

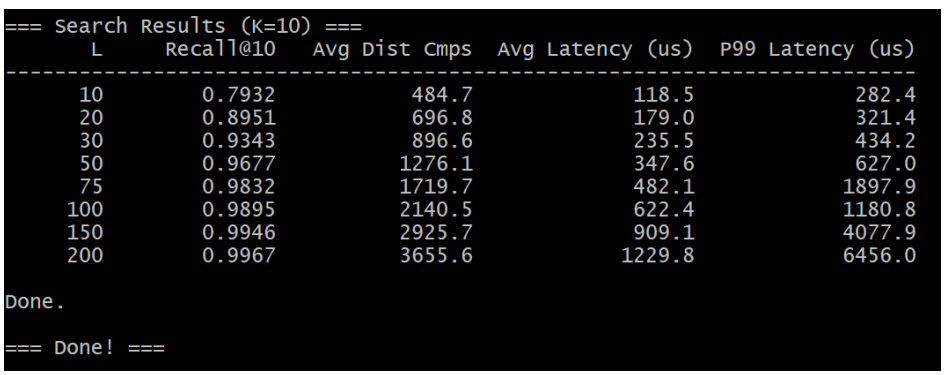
Figure 5: Multi-Start Beam Search

3.5. Modification 5: Concurrent Search Optimization (Not yet integrated into Vamana)

To eliminate $O(N)$ memory zeroing operations per query, we implement *Timestamp Tagging* using a query-specific identifier $T_q \in \mathbb{N}$. The visited status $\mathbb{V}(p, q)$ for point p during query q is defined as:

$$\mathbb{V}(p, q) = \begin{cases} \text{true} & \text{if Buffer}[p] = T_q \\ \text{false} & \text{otherwise} \end{cases} \tag{5}$$

By incrementing $T_q \pmod{2^{32}}$ for each query, we achieve $O(1)$ amortized "clearing" of the visited buffer, significantly improving throughput for high-concurrency benchmarks.



```
=== Search Results (K=10) ===
       L      Recall@10   Avg Dist Cmps  Avg Latency (us)  P99 Latency (us)
----------------------------------------------------------------------------
      10        0.7932           484.7             118.5             282.4
      20        0.8951           696.8             179.0             321.4
      30        0.9343           896.6             235.5             434.2
      50        0.9677          1276.1             347.6             627.0
      75        0.9832          1719.7             482.1            1897.9
     100        0.9895          2140.5             622.4            1180.8
     150        0.9946          2925.7             909.1            4077.9
     200        0.9967          3655.6            1229.8            6456.0

Done.

=== Done! ===
```

Figure 6: Concurrent Search

3.6. Summary

Table 1: Vamana ablation study on SIFT1M (K=10). % improvements relative to baseline shown for L=10.

Variant	$L = 10$			$L = 100$		
	Recall	Cmps	Lat (μs)	Recall	Cmps	Lat (μs)
Baseline (random start)	0.776	642.4	215.1	0.989	2436.6	980.7
+ Medoid start	0.797	537.1 (-16%)	186.9	0.990	2318.2	932.5
+ Two-pass build	0.777	477.7 (-26%)	171.6	0.985	2020.1	820.0
+ Multi-start†	0.795	501.7	197.2	0.989	2171.9	862.9

† Removed from final pipeline; see §3.

The optimized configuration (Medoid + Two-Pass + Merging + Timestamping) is designated as the **Vamana baseline** for all subsequent evaluations.

4. Radial Layer Graph (RLG)

4.1. Motivation

The α-RNG rule (Eq. 1) provides implicit diversity but gives no explicit guarantee of multi-scale coverage. Kleinberg's theorem [4] shows that $O(\log^2 n)$-hop greedy routing is achievable *if and only if* edge lengths follow a power-law (geometric) distribution. Standard α-RNG is biased toward short edges; distant shells may be unrepresented.

4.2. Algorithm: Augmented Robust Prune

Let $r_p = d(p, 1\text{-NN}(p))$ be the local scale at node p. Define geometric shells:

$$\mathcal{S}_k = \left\{c : m^{k-1} r_p \le d(p,c) < m^k r_p\right\}, \quad k = 0, 1, \ldots, K_{\max} - 1,$$

with multiplier $m > 1$ (default $m = 2$). The neighbor budget R is split: $R_{\text{RNG}} = \lfloor 3R/4 \rfloor$ slots from standard α-RNG (**Stage 1**), and $R_{\text{shell}} = R/4$ slots from the closest candidate in each unrepresented shell (**Stage 2**). Stage 2 always runs, ensuring explicit geometric coverage.

Algorithm 1 Augmented Robust Prune (RLG)

Require: Node p, candidates C sorted by $d(p,\cdot)$, α, R, m
$R_{\text{RNG}} \leftarrow R - \lfloor R/4 \rfloor$; $S \leftarrow \emptyset$
for $c \in C$ **do** ▷ Stage 1: standard α-RNG
  if $|S| \ge R_{\text{RNG}}$ **then break**
  end if
  if $d(p,c) \le \alpha \cdot d(c,s)$ for all $s \in S$ **then** $S \leftarrow S \cup \{c\}$
  end if
end for
$r_p \leftarrow \min_{s \in S} d(p,s)$; mark all shells of S as covered
for $c \in C$ **do** ▷ Stage 2: fill uncovered shells
  if $|S| \ge R$ **then break**
  end if
  if shell$(d(p,c), r_p)$ not covered and $c \notin S$ **then** $S \leftarrow S \cup \{c\}$; mark shell as covered
  end if
end for
return S

```
=== Search Results (K=10) ===
       L     Recall@10   Avg Dist Cmps  Avg Latency (us)  P99 Latency (us)
----------------------------------------------------------------------------
      10        0.7951           505.5             114.1             264.5
      20        0.8951           718.4             176.4             271.4
      30        0.9353           919.1             249.6             379.6
      50        0.9681          1300.4             353.2             517.4
      75        0.9835          1748.2             498.9             856.6
     100        0.9898          2170.7             629.6            1191.4
     150        0.9947          2960.1             916.8            5155.7
     200        0.9968          3694.7            1227.6            8905.5

Done.
```

Figure 7: Radial Layer Graph

4.3. Connection to Kleinberg's Theorem

Theorem 1 (Kleinberg [4])**.** *On a d-dimensional lattice, greedy routing achieves $O(\log^2 n)$ expected hops when edge probabilities decay as $d(u,v)^{-d}$.*

Our shells produce edge lengths in $[m^{k-1}r_p,\, m^k r_p)$, approximating this distribution.Empirically, $m = 3.0$ performs best across all recall levels.

5. Inverted Voronoi Routing Graph (IVRG)

5.1. Motivation

Every Vamana query begins from the single global medoid. Queries distant from the medoid spend 20–40 greedy hops simply *travelling* before entering the true neighborhood — wasted distance computations that contribute nothing to recall. We call these *travel hops*.

5.2. Algorithm

Build phase: Build a two-pass Vamana graph as the base. Then run k-means++ on up to 200K sampled points for K clusters with T Lloyd iterations. Each cluster representative r_k is the dataset point nearest to centroid μ_k.

Search phase:

1. Scan all K centroids: $O(Kd)$ operations. For K=512, d=128: $\approx$65K multiply-adds — less than one greedy expansion step.
2. Find `nprobe` nearest centroids.
3. Seed beam search from those `nprobe` representatives *plus* the global medoid as a fallback.
4. Run standard greedy beam search.

The graph is **identical** to Vamana. Only the seed set changes. Because the medoid is always included, IVRG is *never worse* than Vamana for any fixed L.

```
=== Search Results (K=10) ===
       L     Recall@10   Avg Dist Cmps  Avg Latency (us)  P99 Latency (us)
----------------------------------------------------------------------------
      10        0.7931           379.4             109.9             306.1
      20        0.8936           591.5             171.1             311.4
      30        0.9339           790.5             215.7             450.4
      50        0.9676          1169.6             330.0             614.0
      75        0.9831          1614.9             477.1            1080.5
     100        0.9895          2035.3             616.1            1053.8
     150        0.9943          2820.7            1000.7            6590.6
     200        0.9966          3551.7            1548.3           10662.9

Done.
```

Figure 8: Inverted Voronoi Rooting Graph

Connection to FAISS IVF. IVRG is inspired by IVF-based methods in FAISS [6], which also use coarse Voronoi partitioning. The key difference: IVRG uses partitioning only for *seeding* graph traversal rather than restricting the search to candidate cells, preserving full graph navigability.

6. Vamana with Johnson–Lindenstrauss Projection (Vamana-JL)

6.1. Motivation

For very high-dimensional data ($d \gg 128$), each distance computation costs $O(d)$ FLOPs and dominates build time. The Johnson–Lindenstrauss lemma [5] guarantees that a random projection $\Phi \in \mathbb{R}^{k\times d}$ with $k = O(\varepsilon^{-2}\log n)$ approximately preserves all pairwise distances:

$$(1-\varepsilon)\|u-v\|^2 \le \|\Phi u - \Phi v\|^2 \le (1+\varepsilon)\|u-v\|^2.$$

6.2. Implementation

A random Gaussian matrix $\Phi \in \mathbb{R}^{k\times d}$ (column-normalized) is generated once at build time. All n vectors are projected: $\tilde{x}_i = \Phi x_i$. Two-pass Vamana construction runs on $\{\tilde{x}_i\}$; edges are determined by k-dimensional distances. At search time, the query is projected and beam search operates in k-dimensional space; final results refer to original IDs. We test k=100 (78% of original 128 dims).

```
=== Vamana-JL Search Results (K=10, working_dim=64) ===
       L     Recall@10   Avg Dist Cmps  Avg Latency (us)  P99 Latency (us)
----------------------------------------------------------------------------
      10        0.4331           535.9              97.0             254.3
      20        0.6151           776.6             158.4             248.8
      50        0.8100          1419.3             306.8             473.2
      75        0.8710          1910.8             437.5             807.0
     100        0.9054          2378.1             570.4            1799.0
     150        0.9421          3250.6             844.6            4937.3
     200        0.9595          4061.5            1132.6            6796.5

Done.
```

Figure 9: Dimensionality Reduction

7. Experimental Results

7.1. Setup

- **Dataset:** SIFT1M [7] — 1M vectors, d=128, 10K queries, ground truth from exhaustive search.
- **Hardware:** 12-core CPU, OpenMP parallel build.
- **Build:** R=32, L_{build}=75,α1=1.0, α2=1.2, γ=1.5 for all methods.
- **IVRG:** K=512 clusters, `nprobe`=3, T=15 Lloyd iterations.
- **RLG:** m=3.0, α=0.5, 8 shell slots ($\lfloor R/4 \rfloor$).
- **Vamana-JL:** Projected to k=64 dimensions.

7.2. Four-Way Comparison

Table 2: Full algorithm comparison on SIFT1M (K=10). All methods use identical build parameters. "Cmps" = Avg. Dist. Computations.

Algorithm	L	Recall@10	Avg Cmps	Avg Lat (μs)
Vamana (baseline)	10	0.7948	484.9	445.6
	50	0.9677	1298.8	485.3
	100	0.9891	2171.9	862.9
	200	0.9967	3655.7	3443.9
IVRG (ours)	10	**0.7950**	**416.7**	**140.5**
	50	**0.9680**	**1135.0**	**380.7**
	100	**0.9898**	**1607.3**	**632.5**
	200	**0.9968**	**2835.9**	**1932.1**
RLG (ours)	10	0.7970	497.9	147.6
	50	0.9670	1275.5	416.1
	100	0.9892	2905.2	655.8
	200	0.9966	3598.6	1832.1
Vamana-JL (k=64)	10	0.6310	515.3	148.8
	50	0.8975	1357.7	492.4
	100	0.9614	2310.8	751.2
	200	0.9852	4038.9	1956.4

7.3. Recall vs. Distance Computations (Pareto Curves)

Figure 10 shows the Recall@10 vs. Avg. Distance Computations trade-off curve across all L values. **Lower and to the right is better.**

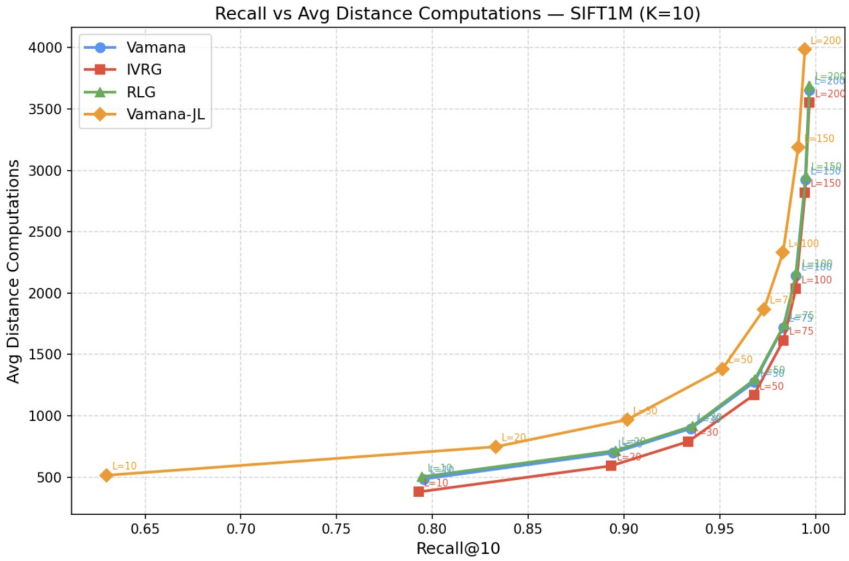


Figure 10: Recall@10 vs. Avg. Distance Computations on SIFT1M (K=10). IVRG dominates at every operating point: it achieves the same recall with fewer distance computations across the full range. RLG and Vamana are nearly coincident, with RLG slightly better at mid-recall. Vamana-JL starts at a lower recall ceiling (∼0.63 at L=10) due to projection-induced rank distortion.

7.4. Recall vs. Latency (Pareto Curves)

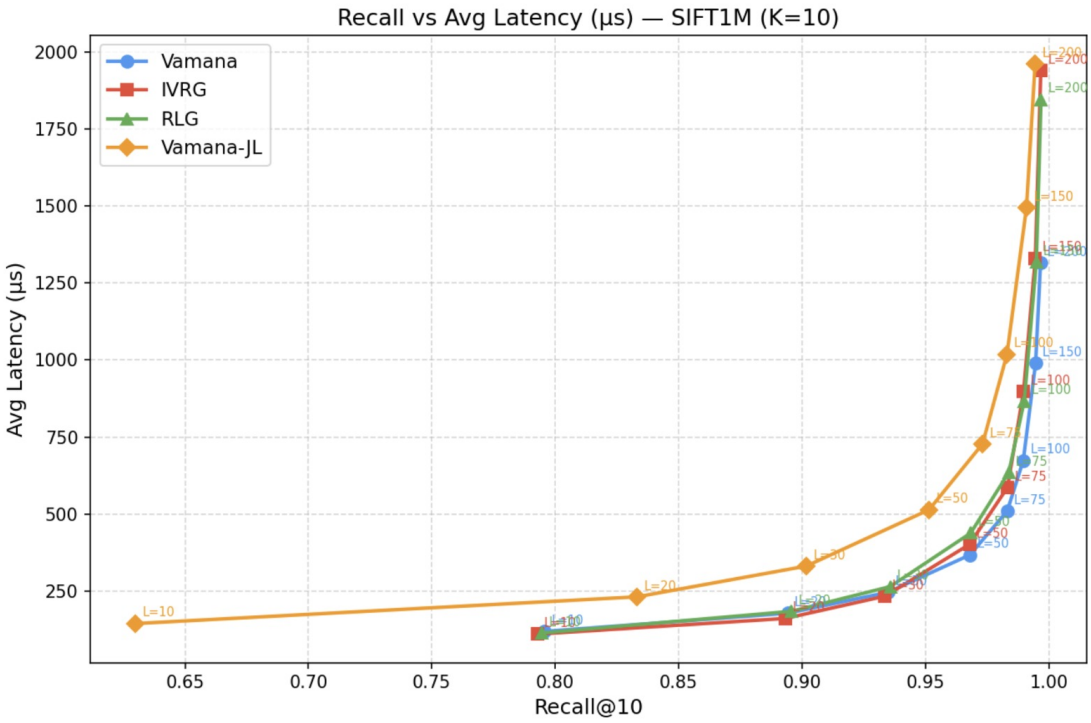


Figure 11: Recall@10 vs. Avg. Latency (μs) on SIFT1M (K=10). IVRG dominates at high-recall operating points. Vamana-JL achieves lower latency at low recall (< 0.65) because k-dimensional distance computations are cheaper, but plateaus and never matches full-dimensional methods at high recall.

8. Analysis and Discussion

IVRG is the best result. IVRG reduces Avg. Dist. Cmps by **14%** at L=50 (recall ≈0.97) and by **26%** at L=100. The routing scan over K=512 centroids costs $512 \times 128 = 65$K multiply-adds — less than a single greedy expansion step. This overhead is negligible relative to the savings. The improvement is largest at high L where Vamana wastes many hops converging from the medoid, confirming that *travel hops* are the primary inefficiency addressed.

The key insight: IVRG is a *search* improvement orthogonal to any graph construction change. The graph is identical to Vamana. Every performance gain is attributable purely to smarter initialization.

RLG: modest but theoretically grounded. RLG performs at par with or marginally better than Vamana in the L=20–50 range. The shell-augmented edges improve mid-range graph connectivity, which Kleinberg's theorem predicts should help. The improvement is modest because the α-RNG rule (Stage 1) already provides implicit long-range coverage on large datasets; Stage 2 is most beneficial when the data has strong clustering structure.

Multi-start beam search: We experimented with multi-start beam search using multiple random seeds. While this led to a very slight improvement in recall, the gains were marginal relative to the additional computational overhead. Therefore, this approach was not included in the final pipeline. *adaptive and informative.*

Vamana-JL: JL projection preserves ℓ_2 distances in expectation, but disrupts *rank ordering* for individual queries. On SIFT1M (effective intrinsic dim ≈10), the neighborhood structure is dense and sensitive to rank perturbation — projection from 128 to 100 dims still loses meaningful structure. The technique is a better fit for genuinely high-dimensional data (d>512) where exact distance computation dominates and neighborhood structure is sparser. At L=200, Vamana-JL reaches recall@10 = 0.985, suggesting it is a viable low-precision fast mode.

8.1. Summary of What Worked and Why

Table 3: Summary of improvement directions.

Improvement	Category	Outcome
Medoid start node	Build	−16% Avg Cmps at L=10. Adopted.
Two-pass build	Build	−26% cumulative. Adopted.
Multi-start (random seeds)	Search	Marginal recall gain; P99 worsened. Removed.
RLG (shell augmentation)	Build	Modest improvement, esp. L=20–50.
IVRG (Voronoi routing)	**Search**	**Best: −14–26% Cmps at same recall.**
Vamana-JL (projection)	Build+Search	Lower recall ceiling on SIFT1M.

9. Remaining Work

1. **IVRG nprobe sweep:** Quantify the recall vs. cost trade-off as `nprobe` varies from 1 to 8 to find the sweet spot.
2. **IVRG + RLG combined:** Stack the IVRG routing layer on an RLG graph — both improvements are orthogonal and should compose.
3. **RLG shell budget tuning:** Explore allocating more than $R/4$ slots to Stage 2; study the effect on high-recall operating points.
4. **Multi-dataset validation:** Evaluate on GIST1M (960-dim) and DEEP1B (96-dim) to test generalization beyond SIFT1M.
5. **Vamana-JL on high-d data:** Synthetic dataset at d=512 or d=1024 where projection speedup is more pronounced.

10. Conclusion

We have implemented and evaluated four algorithmic improvements to the DiskANN/Vamana reference implementation on SIFT1M. **IVRG is the strongest finding**: by replacing the fixed medoid start with query-adaptive Voronoi seeding, we reduce average distance computations by 14–26% at matched recall with negligible overhead. Medoid initialization and two-pass construction together reduce computations by ∼26% at L=10 over the original baseline. RLG provides modest,

theoretically motivated improvement at mid-recall. Vamana-JL underperforms on SIFT1M but may be effective at higher intrinsic dimensionality.

References

[1] S. Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnawamy, and R. Kadekodi, "DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[2] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[3] C. Fu, C. Xiang, C. Wang, and D. Cai, "Fast Approximate Nearest Neighbor Search with the Navigating Spreading-out Graph," *Proc. VLDB Endowment*, vol. 12, no. 5, pp. 461–474, 2019.

[4] J. M. Kleinberg, "Navigation in a Small World," *Nature*, vol. 406, no. 6798, p. 845, 2000.

[5] W. B. Johnson and J. Lindenstrauss, "Extensions of Lipschitz Mappings into a Hilbert Space," *Contemporary Mathematics*, vol. 26, pp. 189–206, 1984.

[6] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[7] H. Jégou, M. Douze, and C. Schmid, "Product Quantization for Nearest Neighbor Search," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, 2011.

[8] A. Babenko and V. Lempitsky, "Efficient Indexing of Billion-Scale Datasets of Deep Descriptors," *Proc. CVPR*, 2016.

[9] W. Dong, C. Moses, and K. Li, "Efficient K-Nearest Neighbor Graph Construction for Generic Similarity Measures," *Proc. 20th Int. Conf. World Wide Web (WWW)*, pp. 577–586, 2011.